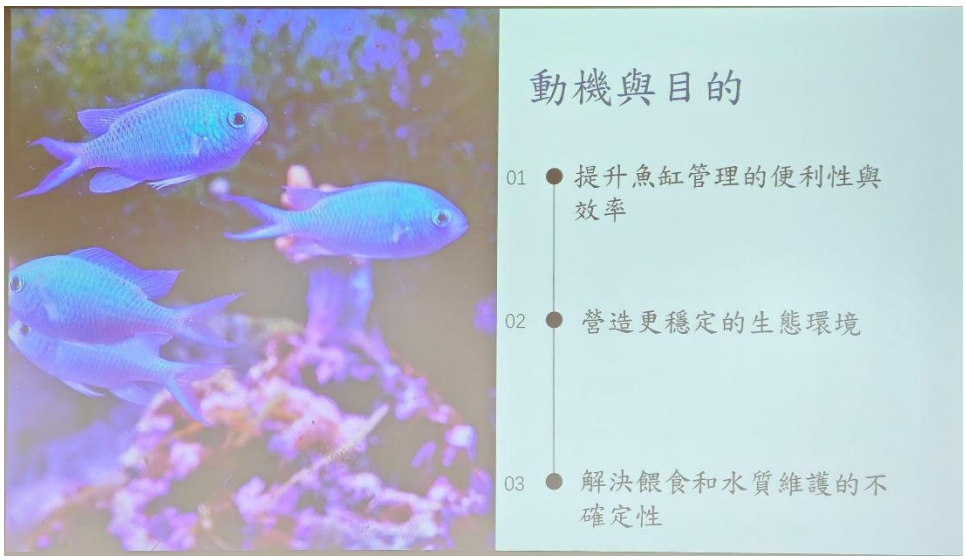
日期:2026/03/10

演講者:陳永隆教授

題目**智慧系統設計與應用**

第一主題

智慧魚缸系統



動機與目的

1. 提升魚缸管理的便利性與效率

把原本要人工做的事情，例如查看水溫、開燈、餵食、加熱、抽水，交給系統協助處理，減少人力負擔。

2. 營造更穩定的生態環境

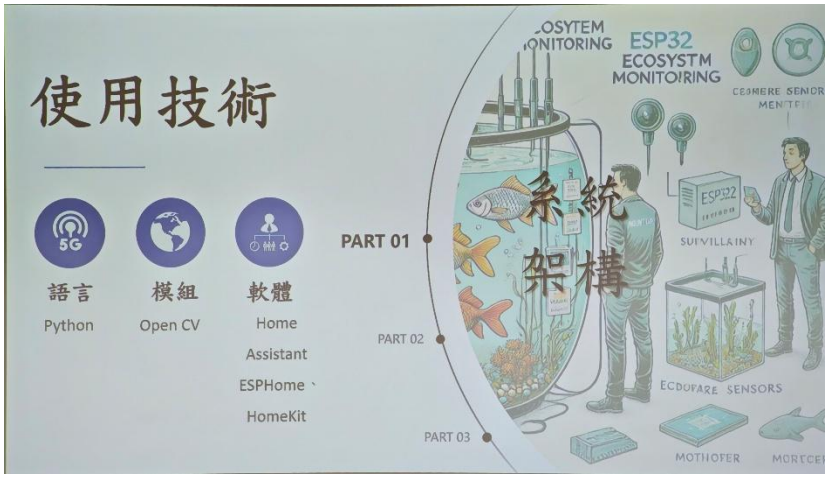
透過感測器持續監測魚缸狀態，讓魚缸環境維持在較適合魚類生存的範圍內。

3. 解決餵食與水質維護的不確定性

將定時餵食、混濁度偵測、提醒換水等功能整合進系統，避免因忘記照顧或判斷不準造成問題。

4. 建立可遠端操作的智慧魚缸平台

除了現場硬體控制，也能在電腦端、手機端即時查看數據與操作設備。



使用技術

這套系統主要整合

1. Python

作為主要程式開發語言，用來做後端控制、資料處理與影像辨識流程。

2. OpenCV

用於影像辨識模組，進行魚種辨識，讓系統能根據辨識結果做後續調整或顯示。

3. ESP32

作為整個硬體控制核心，負責連接各種感測器與控制元件，並將資料送往上層平台。

4. ESPHome

作為 ESP32 與智慧家庭平台之間的整合橋樑，讓硬體資料與裝置控制更容易接入系統。

5.MQTT

作為 OpenCV 與 Home Assistant 之間的資料傳遞機制

6.感測與控制硬體

從實體照片中能看到他們整合溫度感測器

水質／混濁度感測器、光照感測器、LCD 顯示器、RGB 燈光、繼電器模組、加熱器、抽水馬達、餵食裝置、實體按鈕、攝影機／監視器

在控制功能非常充足

系統功能

系統功能大致上可以分成**環境監測、設備控制、介面、影像辨識**四種大方向

電腦控制介面

環境監測功能

溫度感測

系統持續監測魚缸水溫。
當水溫低於設定臨界值時，會啟動加熱裝置，使水溫回到安全範圍。

混濁度／水質感測

系統透過水質或混濁度感測器判斷魚缸水體是否變差。
若低於設定標準，系統會提醒使用者換水，以維持水體乾淨。

光照感測

當外界光線不足時，系統可自動開啟 LED 燈光，補足魚缸照明需求。

自動遠端控制功能

餵食系統

可以：

- 設定魚類的餵食時間
- 使用網頁按鈕隨時餵食

加熱控制

當溫度不足時自動啟動加熱器，也可以透過介面手動開關。

燈光控制

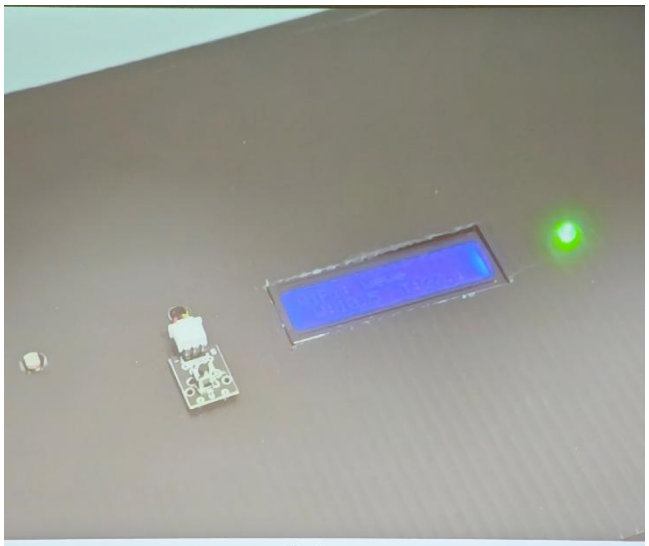
可依環境光線自動補光，也可手動控制燈光開關。

抽水／換水控制

系統可搭配水質狀況進行提醒，並透過抽水裝置執行換水相關操作。

顯示介面

他們做了



外觀

LCD 顯示

裝置本體顯示

控制介面

從上方投影片可看出介面顯示

手機控制介面

影像辨識功能

這部分設計兩種功能

魚缸即時影像監視

透過攝影機持續取得魚缸畫面，並可在平台端即時顯示。

魚種辨識

系統透過 **OpenCV** 執行影像辨識，判斷魚的種類。

亮點

報告的亮點，我認為是 五專生做得但是整體軟硬體整合度很高，不是單一功能作品，而是整合很多感覺確實不錯的功能我覺得比較好的部分是能降低養魚管理的人工作業量、提高魚缸環境穩定度、增加遠端監控與操作便利性。

第二主題

口罩檢測系統

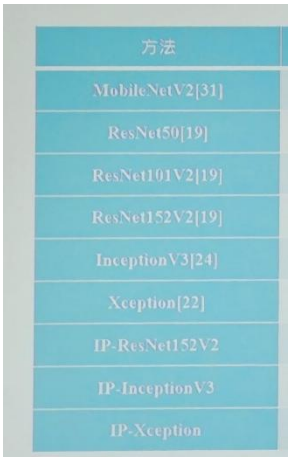
目的

生產線品質檢測、可辨識瑕疵

裁切瑕疵
邊條斷裂
掛繩斷裂
邊條過長
沒有掛繩
正常口罩

這部分演講教授將重點放在數據結果的呈現上

比較不同深度學習模型的辨識效果



方法
MobileNetV2[31]
ResNet50[19]
ResNet101V2[19]
ResNet152V2[19]
InceptionV3[24]
Xception[22]
IP-ResNet152V2
IP-InceptionV3
IP-Xception

報告中提出三種主要方法：

IP-ResNet152V2

IP-InceptionV3

IP-Xception

也有拿原始版本模型做比較，例如：

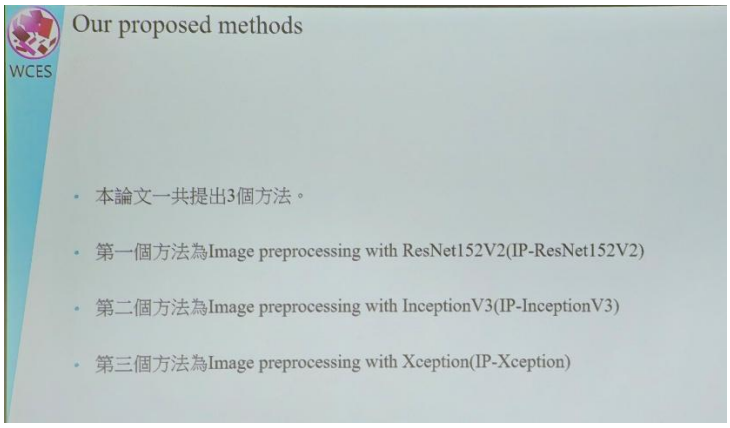
ResNet152V2

InceptionV3

Xception

以及其他對照模型如 **MobileNetV2、ResNet50、ResNet101V2**

使用技術



技術主要分成兩大塊：影像預處理與深度學習分類

(1) ResNet152V2

一種很深的卷積神經網路模型，擅長處理影像分類問題。

在這篇報告裡，搭配圖片預處理後形成 **IP-ResNet152V2**。

(2) InceptionV3

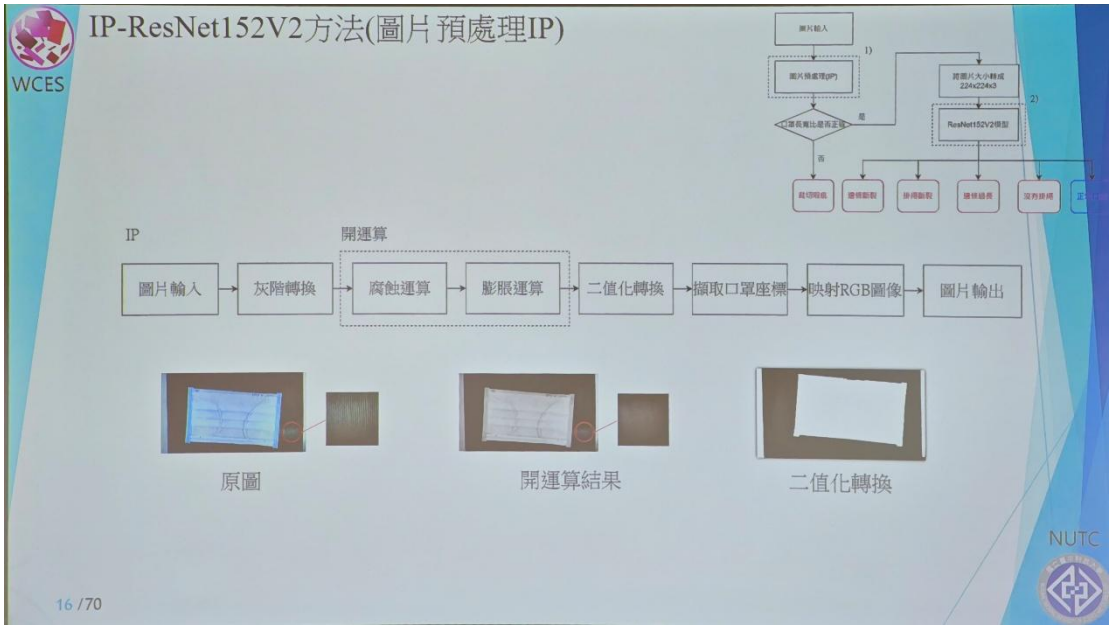
透過多尺度卷積特徵擷取影像資訊。

搭配預處理後形成 **IP-InceptionV3**。

(3) Xception

是 **Inception** 架構的延伸版本，利用 **depthwise separable convolution** 提升效果。

搭配預處理後形成 **IP-Xception**。



圖片預處理

從 **IP-ResNet152V2** 的「圖片預處理 **IP**」投影片可看出，流程大致包含：

圖片輸入

灰階轉換

開運算

腐蝕運算

膨脹運算

二值化轉換

擷取口罩座標

映射 **RGB** 圖像

圖片輸出

Loss Function	categorical_crossentropy
學習率	0.0001
訓練回合數	15、30、60
Batch size	16

訓練設定都是用

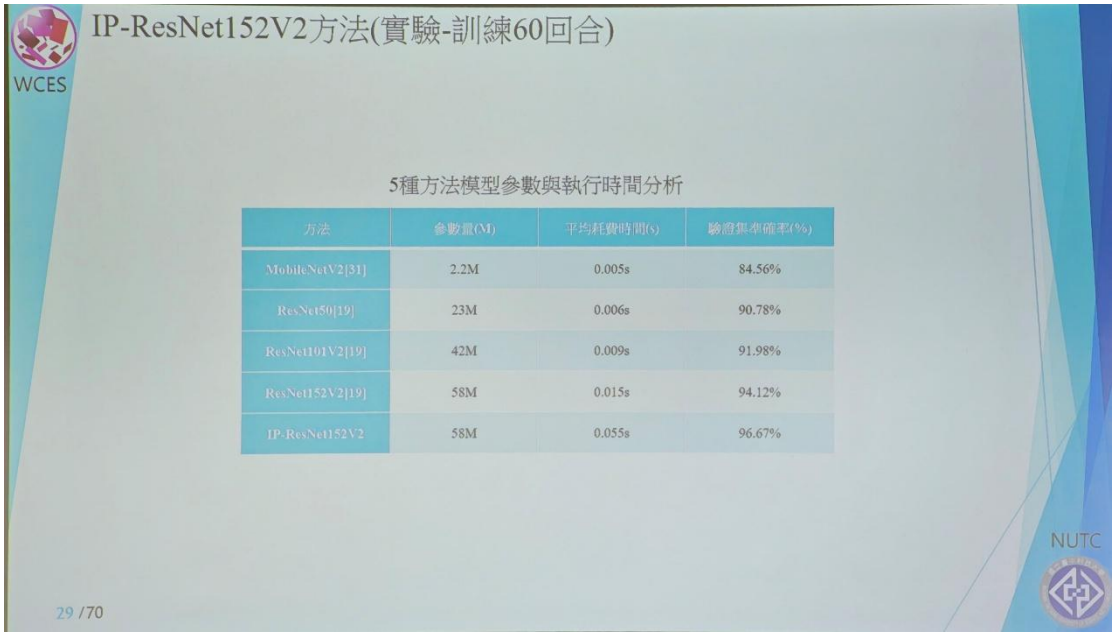
Loss Function：categorical_crossentropy

學習率：**0.0001**

訓練回合數：**15、30、60**

Batch size：16

實驗結果



IP-ResNet152V2 方法

方法特色

- 先做圖片預處理

- 再交給 **ResNet152V2** 模型分類
- 輸入大小為 **224×224×3**

實驗結果

訓練 **30** 回合

- 訓練集準確率：**96.03%**
- 驗證集準確率：**94.56%**
- 訓練集 **loss**：**0.15**
- 驗證集 **loss**：**0.21**

訓練 **60** 回合

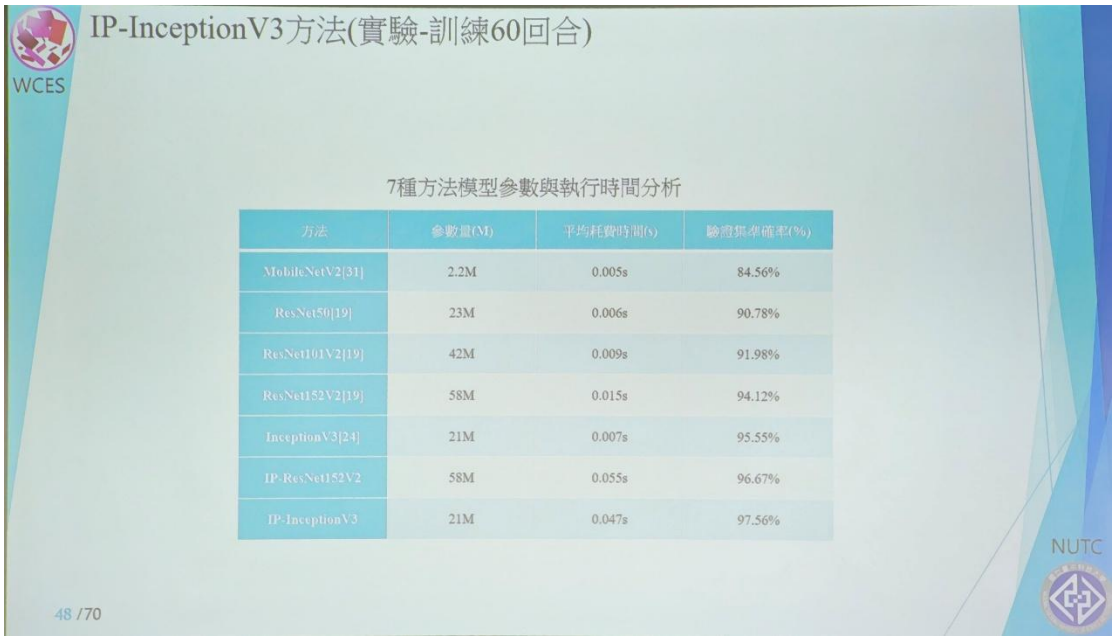
- 訓練集準確率：**97.29%**
- 驗證集準確率：**96.67%**
- 訓練集 **loss**：**0.05**
- 驗證集 **loss**：**0.15**

與原始模型比較

表格顯示：

- **ResNet152V2**：**94.12%**
- **IP-ResNet152V2**：**96.67%**

加入圖片預處理後，驗證準確率提升了 **2.55%**。



方法	參數量(M)	平均耗費時間(s)	驗證集準確率(%)
MobileNetV2[31]	2.2M	0.005s	84.56%
ResNet50[19]	23M	0.006s	90.78%
ResNet101V2[19]	42M	0.009s	91.98%
ResNet152V2[19]	58M	0.015s	94.12%
InceptionV3[24]	21M	0.007s	95.55%
IP-ResNet152V2	58M	0.055s	96.67%
IP-InceptionV3	21M	0.047s	97.56%

2. IP-InceptionV3 方法

方法特色

- 先做圖片預處理
- 再交給 **InceptionV3** 模型
- 輸入大小為 **299×299×3**

實驗結果

訓練 **30** 回合

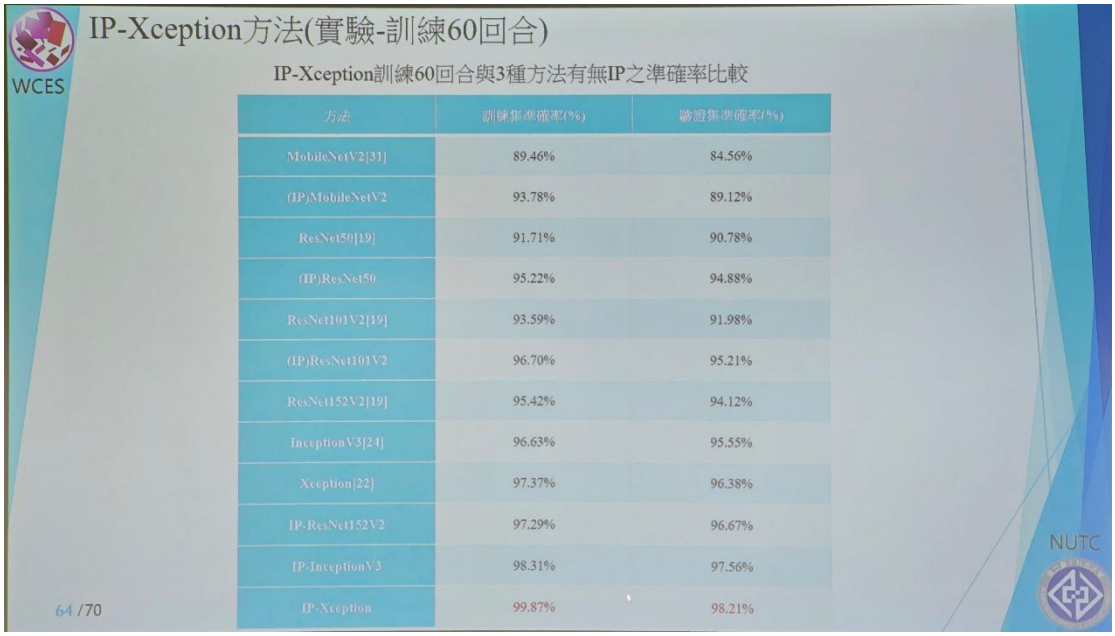
- 訓練集準確率：**97.23%**
- 驗證集準確率：**96.83%**
- 訓練集 **loss**：**0.25**
- 驗證集 **loss**：**0.13**

訓練 **60** 回合

從總表與結論可讀出：

- **InceptionV3**：**95.55%**
- **IP-InceptionV3**：**97.56%**

表示加入圖片預處理後，驗證準確率提升 **2.01%**。



方法	訓練集準確率(%)	驗證集準確率(%)
MobileNetV2[31]	89.46%	84.56%
(IP)MobileNetV2	93.78%	89.12%
ResNet50[19]	91.71%	90.78%
(IP)ResNet50	95.22%	94.88%
ResNet101V2[19]	93.59%	91.98%
(IP)ResNet101V2	96.70%	95.21%
ResNet152V2[19]	95.42%	94.12%
InceptionV3[24]	96.63%	95.55%
Xception[22]	97.37%	96.38%
IP-ResNet152V2	97.29%	96.67%
IP-InceptionV3	98.31%	97.56%
IP-Xception	99.87%	98.21%

3. IP-Xception 方法

方法特色

- 先做圖片預處理
- 再用 **Xception** 做分類
- 是這篇報告表現最好的方法

實驗結果

訓練 **15** 回合

- 訓練集準確率：**98.76%**
- 驗證集準確率：**97.88%**
- 訓練集 **loss**：**0.01**
- 驗證集 **loss**：**0.09**

訓練 **30** 回合

- 訓練集準確率：**99.57%**
- 驗證集準確率：**97.97%**

- 訓練集 **loss：0.02**
- 驗證集 **loss：0.07**

訓練 **60** 回合

結論顯示：

- **Xception：96.38%**
- **IP-Xception：98.21%**

加入圖片預處理後，驗證準確率提升 **1.83%**。

結果統整

1. 不加 **IP** 的最佳結果

從比較表看，原始模型中表現最好的是：

Xception：96.38%

2. 加入 **IP** 後的最佳結果

表現最好的是：

IP-Xception：98.21%

3. IP 的整體效果

三個主要方法都因為加入 **IP** 而提升：

ResNet152V2：+2.55%

InceptionV3：+2.01%

Xception：+1.83%

所以圖片預處理確實能有效提升口罩瑕疵分類的驗證準確率。

這份研究主要針對口罩檢驗問題，提出結合圖片預處理與深度學習模型的自動分類方法。研究中分別將預處理技術結合 **ResNet152V2**、**InceptionV3** 與 **Xception**，結果顯示加入預處理後，三種方法的驗證準確率皆有提升，其中以 **IP-Xception** 表現最佳，驗證準確率達 **98.21%**。這研究透過先擷取口罩有效區域、降低背景干擾，可有效提升口罩瑕疵辨識能力，具有應用於自動化品檢系

統的潛力。

第三主題

改良 **YOLOv8** 模型之遙測影像目標偵測

主要針對遙測影像中的小目標、多尺度目標、密集目標偵測問題，改良 **YOLOv8n**，提出不同架構版本，並比較偵測效果。

使用技術

YOLO 的基本架構包括：

Backbone

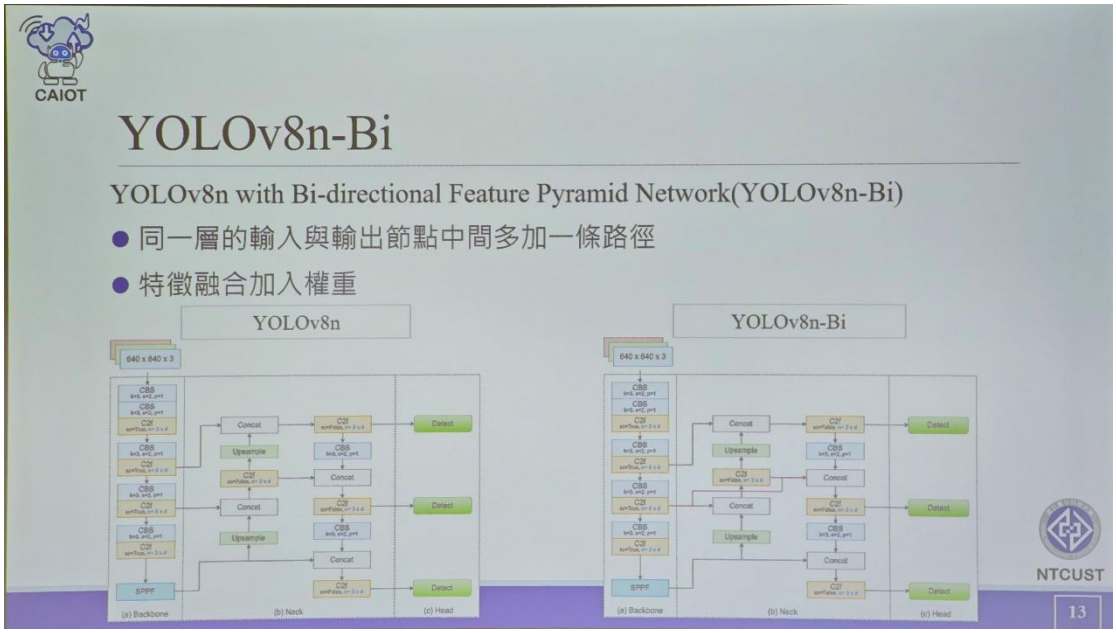
負責提取影像特徵。

Neck

負責融合來自 **Backbone** 的多層特徵圖。

Head

根據 **Neck** 輸出的特徵，做最終的類別與邊界框預測。



Bi-directional Feature Pyramid Network（BiFPN / 雙向特徵金字塔）

特色是同一層的輸入與輸出節點中間多加一條路徑

特徵融合加入權重

這表示模型不只是單純把不同層的特徵 **concat** 起來，而是更有效率地做雙向多尺度特徵融合，讓高層語意資訊與低層細節資訊更好地交換。

架構說明

三個改良版本的架構差異

A. 原始 **YOLOv8n** 架構

YOLOv8n 的結構可以拆成三部分：

1. Backbone

提取影像中的特徵。

2. Neck

融合不同層級的特徵圖。

3. Head

根據特徵圖輸出最終的類別與邊界框。

這是一個典型單階段物件偵測架構，速度快、計算量相對小。

B. YOLOv8n-Bi

全名：**YOLOv8n with Bi-directional Feature Pyramid Network**

架構改良重點

同一層的輸入與輸出節點中間多加一條路徑

特徵融合加入權重

原本 **YOLOv8n** 的 **Neck** 已經有多尺度融合，但 **YOLOv8n-Bi** 進一步讓資訊可以更完整地在不同層之間雙向流動，提升特徵融合品質。

目的改善：

小目標偵測

多尺度目標偵測

特徵融合效率

C. YOLOv8n-TFBi

這個版本是在 **YOLOv8n-Bi** 之上，再加入 **Transformer**。

架構改良重點

保留雙向特徵金字塔

加入 **Transformer** 模組

目的

強化全域資訊建模能力，讓模型不只看局部，而能更好理解整體場景。

對遙測影像這種大範圍場景特別有意義。

D. YOLOv8n-TFBiCA

全名：**YOLOv8n with Transformer and Bi-directional Feature Pyramid Network and Coordinate Attention**

這是三個改良版本裡最完整的版本。

架構改良重點

有 **Transformer**

有雙向特徵金字塔

再加入 **Coordinate Attention**

座標注意力的作用

嵌入位置資訊

明確關注特徵圖特定區域

強化空間定位能力